ENGR 1050
Homework 3

Due: Friday, October 16, 2026 by 11:59 PM

Instructions

At this point you should have a command of Python fundamentals. In lecture on Monday 10/12 we step through how to build a simple ODE solver class. Your task for this homework is to use that class to solve a some simple ODE problems. When you approach any numerical problem, you should follow this cookbook:

1. Identify the initial conditions and parameters.
2. Write out the analytic solution, or estimate asymptotic behavior to verify your numerical solution.
3. Use the ODE solver class to solve the problem numerically, increasing N until the solution has converged.
4. Compare the numerical solution to the analytic solution.
5. Plot the results, comparing the numerical and analytic solutions.

For some problems, you will not be able to find an analytic solution. In these cases, you can use other qualitative methods to verify your solution (e.g. checking asymptotic/steady-state behavior).

- Your Jupyter notebook must include for each problem: a statement of the ODE, your code, a plot showing that your code has *converged* (i.e. a plot of solutions for $N = 2, 4, 8, ...$ until the solution no longer changes), and a plot comparing the numerical solution against the analytic solution.
- Submit your solutions as an uploaded Jupyter notebook (.ipynb) to the appropriate assignment on Canvas, along with screenshots of your final results to show your code is working.
- Complete this assignment on your own. To comply with UPenn's academic integrity policy, you must include a comment in the code stating all references you used to complete the assignment, including any help you received from classmates, and whether you used AI (we suggest in the strongest possible terms that you don't).
- Remember: Late submissions will not be accepted without documentation. The lowest homework score will be dropped.

Problems

Problem 1: Skydiver (50 points)

The motion of a skydiver can be modeled by the following ordinary differential equation (ODE):

$$m\frac{dv}{dt} = mg - kv^2 \tag{1}$$

where m is the mass of the skydiver, g is the acceleration due to gravity, k is a drag coefficient, and v is the velocity of the skydiver.

(a) (5 points) Identify the initial condition for a skydiver with a mass of 80 kg, a drag coefficient of 15 kg/m, and an initial velocity of 0 m/s. Assume $g = 9.81$ m/s^2.

(b) (10 points) Identify the *terminal velocity* of the skydiver where the acceleration of the skydiver is zero.

(c) (20 points) Use the ODE solver class to solve the ODE numerically for the skydiver over a time period of 0 to 60 seconds. Plot the numerical solution and compare it to the analytic steady state.

(d) (15 points) If one were to double the drag coefficient, how would that affect the terminal velocity? Run a simulation to verify your answer.

Problem 2: Harmonic Oscillator (50 points)

The motion of a mass-spring system can be modeled by the following ODE:

$$\frac{dx}{dt} = v \tag{2}$$

$$m\frac{dv}{dt} = -kx \tag{3}$$

where m is the mass of the object, k is the spring constant, and x is the displacement of the object from its equilibrium position. We will assume at the initial time $t = 0$ that the mass has an initial displacement of $x(0) = 0$ m and an initial velocity of $v(0) = 1$ m/s. Let $m = 1$ kg and $k = 1$ N/m.

For the following, we will need to measure the error between the true solution and the numerical solution. The following code will compute the norm between two numpy arrays, measuring how large the difference is:

```
import numpy as np

# Define two numpy arrays
array1 = np.array([1, 2, 3])
array2 = np.array([4, 5, 6])

# Compute the norm of the difference between the two arrays
norm = np.linalg.norm(array1 - array2)

print("The norm between the two arrays is:", norm)
```

(a) (10 points) Show that the ODE and initial conditions are satisfied by $x(t) = \sin(t)$.

(b) (20 points) Use the code in class to solve the ODE from $t = 0$ to $t = 2\pi$ seconds. Plot the numerical solution and compare it to the analytic solution, taking $N = 2, 4, 8, 16...$ until they line up.

(c) (20 points) Compute what N is required to achieve an error norm of less than 0.5. Repeat for $k = 1, 2, 4, 8$ N/m. Summarize your findings in markdown.